Seminario

Sistema de gestión de pedidos y producción para una mueblería

Alumno: Ferreira Lautaro
Período: Segundo cuatrimestre de 2026.

Índice

1. Introducción
2. Justificación
3. Definición del proyecto y del sistema
4. Elicitación de requerimientos
   4.1 Requerimientos funcionales
   4.2 Requerimientos no funcionales
5. Conocimiento del negocio
6. Propuesta de solución
7. Casos de uso iniciales
8. Diagramas
9. Etapa de análisis
10. Etapa de diseño
11. Etapa de implementación
12. Etapa de pruebas
13. Base de datos
14. Implementación de la base de datos MySQL
15. Definiciones de comunicación
16. Conclusiones de la AP2
17. Desarrollo del prototipo en Java
18. Aplicación de programación orientada a objetos
19. Menú, estructuras de control y excepciones
20. Pruebas de funcionamiento y evidencia
21. Conclusiones de la AP3
22. Integración final del proyecto: AP4
23. Patrón de diseño seleccionado y justificación
24. Interfaces, abstracción y reutilización
25. Arreglos y ArrayList complementarios
26. Persistencia MySQL y gestión de excepciones
27. Pruebas y resultados
28. Alcance y limitaciones del prototipo final
29. Presentación del desarrollo y video
30. Conclusiones de la AP4
31. Evidencias de ejecución de AP4
32. Diagrama de arquitectura del prototipo
33. Publicación y demostración final
34. Referencias

1. Introducción

La gestión de pedidos en una mueblería dedicada a la fabricación a medida requiere administrar información de clientes, presupuestos, pagos, producción y entregas. Cuando estos datos se encuentran distribuidos entre planillas, mensajes y registros manuales, pueden producirse errores, pérdida de información y dificultades para conocer el estado real de cada trabajo.

El presente proyecto propone desarrollar un sistema informático basado en Java y MySQL que permita centralizar los principales procesos administrativos y productivos de una mueblería, mejorando el seguimiento de los pedidos desde su registro hasta su entrega.

2. Justificación

Actualmente, la información correspondiente a clientes, pedidos y pagos puede quedar distribuida en diferentes herramientas, lo que dificulta obtener una visión completa de cada operación. La centralización de estos datos permitirá mejorar la trazabilidad de los trabajos y disponer de información actualizada para la gestión.

El sistema permitirá:

- Centralizar la información de clientes y pedidos.
- Conocer rápidamente el estado de cada trabajo.
- Registrar los pagos realizados y calcular los saldos pendientes.
- Mejorar la comunicación entre administración y producción.
- Conservar un historial de pedidos y pagos.

La propuesta busca reducir errores administrativos y facilitar la toma de decisiones. Esto se relaciona con el rol de los sistemas de información transaccionales, que permiten registrar y administrar las operaciones habituales del negocio (Saroka, 2002).

3. Definición del proyecto y del sistema

Objetivo general

Desarrollar un sistema informático que permita gestionar clientes, pedidos, pagos y estados de producción de una mueblería de fabricación a medida.

Objetivos específicos

- Centralizar la información de clientes y pedidos.
- Registrar pagos y calcular saldos pendientes.
- Permitir el seguimiento del estado de producción.
- Facilitar la consulta de información histórica.

Alcance

El sistema permitirá registrar y modificar clientes, crear pedidos, registrar pagos, consultar saldos, consultar pedidos según su estado, actualizar el avance de producción y registrar la entrega de los trabajos.

No incluirá inicialmente facturación electrónica, gestión automática de stock, pagos electrónicos ni integración con maquinaria industrial.

Aplicación del PUD

El proyecto se desarrolló siguiendo el Proceso Unificado de Desarrollo (PUD), mediante un enfoque iterativo e incremental. En la fase de Inicio se definieron el problema, el alcance y los requerimientos principales. Durante la Elaboración se profundizaron los casos de uso, la arquitectura y el modelo de datos. En Construcción se implementaron progresivamente los módulos de clientes, pedidos y pagos utilizando Java y MySQL. Finalmente, en Transición se realizaron pruebas funcionales, correcciones y verificaciones del prototipo.

4. Elicitación de requerimientos

La elicitación busca adquirir el conocimiento necesario para comprender el dominio del problema y definir los requerimientos del sistema. Para este proyecto se utilizarán entrevistas, observación y análisis de documentación, técnicas compatibles con las propuestas en la lectura del módulo y en Kendall y Kendall (2011).

Las entrevistas estarán dirigidas al propietario, personal administrativo y responsable de producción. Algunas preguntas serán:

- ¿Cómo se registran actualmente los pedidos?
- ¿Cómo se controla cuánto abonó cada cliente?
- ¿Cómo se conoce el estado de fabricación de un pedido?
- ¿Qué información se consulta con mayor frecuencia?
- ¿Qué problemas se producen actualmente?

También se observará el proceso desde la aceptación de un presupuesto hasta la entrega del producto y se analizarán documentos utilizados en la operatoria, como presupuestos, registros de pagos y listados de trabajos.

4.1 Requerimientos funcionales

RF01 - El sistema debe permitir iniciar sesión mediante usuario y contraseña.

RF02 - El sistema debe permitir registrar y modificar clientes.

RF03 - El sistema debe permitir registrar pedidos asociados a clientes.

RF04 - El sistema debe permitir consultar los pedidos registrados.

RF05 - El sistema debe permitir registrar pagos asociados a un pedido.

RF06 - El sistema debe calcular automáticamente el saldo pendiente de cada pedido.

RF07 - El sistema debe permitir consultar el historial de pagos de un pedido.

RF08 - El sistema debe permitir modificar el estado de producción de un pedido.

RF09 - El sistema debe permitir consultar pedidos según su estado.

RF10 - El sistema debe permitir registrar la entrega de un pedido terminado.

Los requerimientos anteriores corresponden al sistema completo definido durante el análisis. El prototipo operacional desarrollado en las actividades AP3 y AP4 implementa un subconjunto funcional centrado en el registro y consulta de clientes y pedidos, la gestión de pagos, el cálculo de saldos y la actualización de estados. La autenticación, la modificación de datos de clientes y los filtros específicos por estado quedan contemplados como ampliaciones futuras.

4.2 Requerimientos no funcionales

RNF01 - El sistema deberá desarrollarse en Java.

RNF02 - La base de datos deberá implementarse en MySQL.

RNF03 - La conexión entre Java y MySQL se realizará mediante JDBC.

RNF04 - El acceso deberá requerir autenticación mediante usuario y contraseña.

RNF05 - La interfaz deberá ser simple, clara y comprensible para los usuarios.

RNF06 - El sistema deberá mantener la integridad y consistencia de los datos mediante claves primarias, claves foráneas y validaciones.

5. Conocimiento del negocio

El proceso principal comienza cuando un cliente solicita un presupuesto para un mueble. Una vez aceptado, se registra el pedido junto con su importe total y fecha estimada de entrega. El cliente puede realizar uno o varios pagos durante el proceso.

Posteriormente, el pedido pasa al área de producción, donde su estado se modifica según el avance del trabajo. Los estados principales serán: Presupuesto → Confirmado → En producción → Terminado → Entregado.

Los principales actores del negocio son el cliente, el personal administrativo, el responsable de producción y el administrador. El flujo de información comienza con la solicitud del cliente, continúa con la confirmación del pedido y sus pagos, pasa al proceso de fabricación y finaliza con la entrega. El sistema actuará como vínculo entre las áreas administrativa y productiva, manteniendo centralizada la información durante todo el ciclo del pedido.

El problema principal identificado es la falta de centralización de la información, que dificulta conocer rápidamente el estado de los pedidos, los pagos realizados y los saldos pendientes.

6. Propuesta de solución

Arquitectura

- Prototipo desarrollado en Java utilizando una IDE de desarrollo.
- Base de datos MySQL para la persistencia de la información.
- Conexión entre la aplicación y la base de datos mediante JDBC.
- Arquitectura organizada en capas para separar interfaz, lógica de negocio y acceso a datos.

Prototipo operacional

El alcance inicialmente previsto comprendía:

- Gestión de clientes: registro, consulta y modificación.
- Registro y consulta de pedidos.
- Registro e historial de pagos.
- Cálculo automático del saldo pendiente.
- Consulta y actualización del estado de los pedidos.
- Registro de la entrega de pedidos terminados.

La información se almacenará inicialmente en tres entidades principales: Cliente, Pedido y Pago. MySQL permitirá mantener la persistencia e integridad referencial mediante claves primarias y foráneas.

Entidades principales

Entidad	Atributos principales
Cliente	id_cliente (PK), nombre, teléfono, email, dirección
Pedido	id_pedido (PK), id_cliente (FK), fecha, descripción, precio_total, estado
Pago	id_pago (PK), id_pedido (FK), fecha, importe

Relaciones: Cliente 1 - N Pedido; Pedido 1 - N Pago.

Repositorio GitHub: https://github.com/LautaroFR/seminario-ap1-muebleria

7. Casos de uso iniciales

CU01 - Registrar / modificar cliente

Actor: Administrativo.

Descripción: Permite ingresar un nuevo cliente o actualizar sus datos.

CU02 - Registrar pedido

Actor: Administrativo.

Descripción: Permite crear un pedido y asociarlo a un cliente.

CU03 - Registrar pago

Actor: Administrativo.

Descripción: Permite registrar un pago asociado a un pedido y actualizar su saldo pendiente.

CU04 - Consultar pedidos

Actor: Administrativo / Administrador / Responsable de producción.

Descripción: Permite visualizar los pedidos registrados y sus estados.

CU05 - Actualizar producción

Actor: Responsable de producción.

Descripción: Permite modificar el estado de fabricación de un pedido.

CU06 - Consultar saldo

Actor: Administrativo / Administrador.

Descripción: Permite consultar el importe total, los pagos realizados y el saldo pendiente de un pedido.

CU07 - Registrar entrega

Actor: Administrativo / Administrador.

Descripción: Permite registrar la entrega de un pedido terminado y modificar su estado a Entregado.

8. Diagramas

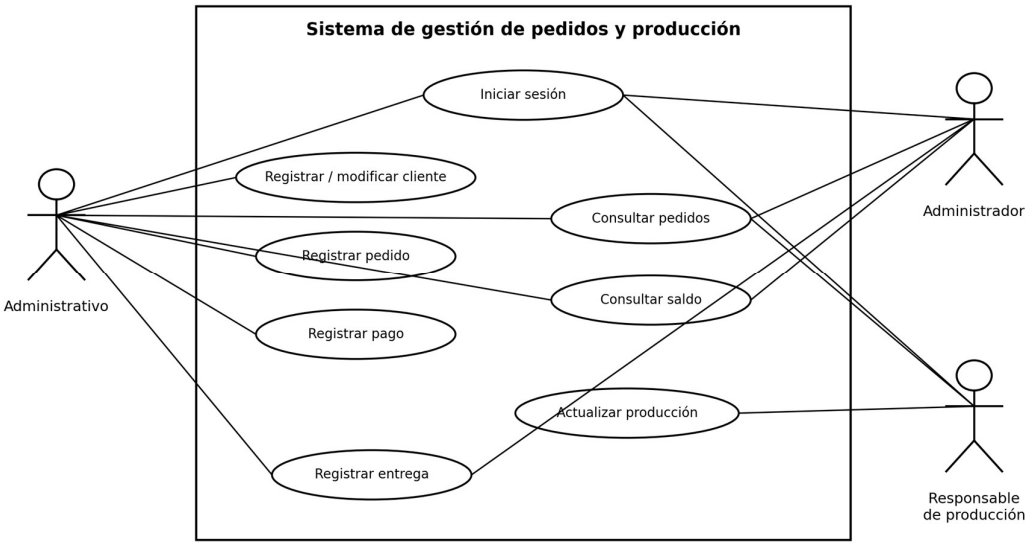


Figura 1 - Diagrama de casos de uso

Figura 1 - Diagrama de casos de uso. Fuente: elaboración propia.

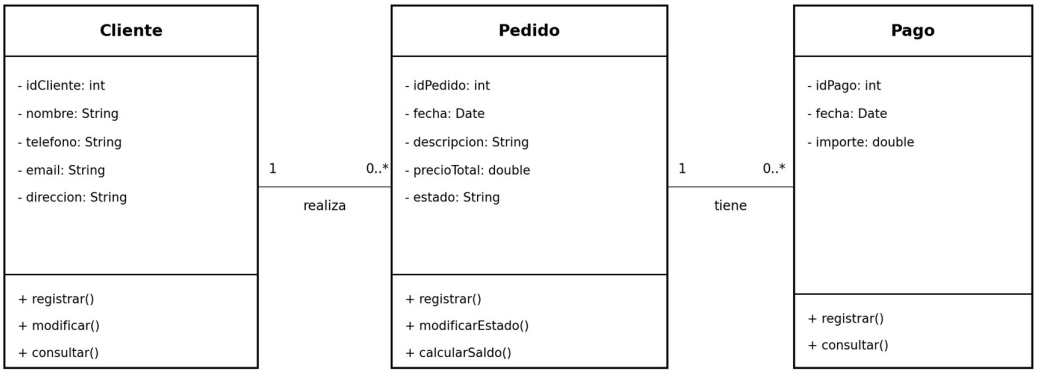


Figura 2 - Diagrama de clases

Figura 2 - Diagrama de clases. Fuente: elaboración propia.

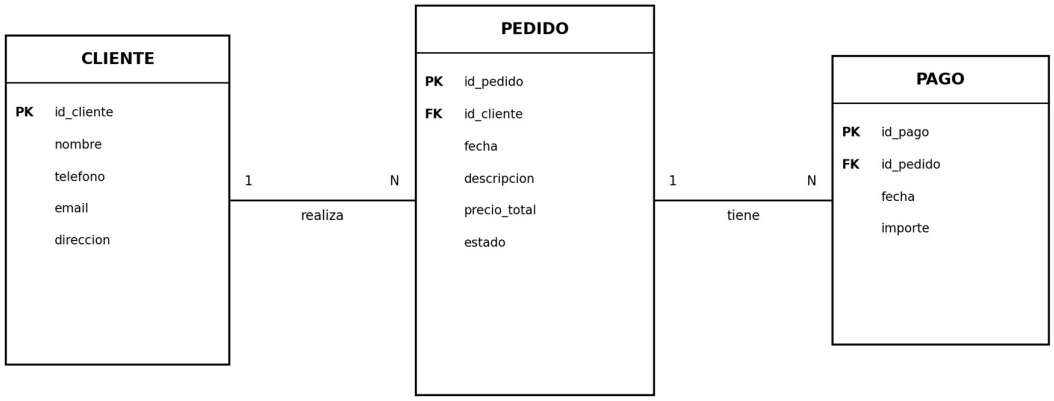


Figura 3 - Modelo Entidad-Relación

Figura 3 - Modelo Entidad-Relación. Fuente: elaboración propia.

9. Etapa de análisis

La etapa de análisis tuvo como objetivo profundizar el estudio realizado en la Actividad Práctica 1 para comprender el funcionamiento de la organización y definir con mayor precisión los procesos que serán soportados por el sistema. Durante esta etapa se identificaron los actores involucrados, el flujo de información y las principales necesidades de cada área de la mueblería.

El análisis confirmó que la principal problemática de la empresa es la dispersión de la información entre planillas de cálculo, registros manuales y conversaciones de mensajería, lo que dificulta conocer el estado de cada pedido, controlar los pagos realizados y realizar el seguimiento de la producción. El sistema propuesto permitirá centralizar toda esta información en una única base de datos, mejorando la trazabilidad y reduciendo errores administrativos.

9.1 Actores del sistema

Actor	Función
Cliente	Solicita presupuestos, confirma pedidos, realiza pagos y recibe el producto terminado.
Administrativo	Registra clientes, pedidos, pagos y entregas.

Actor	Función
Responsable de Producción	Consulta los pedidos asignados y actualiza el estado de fabricación.
Administrador	Administra usuarios, consulta reportes y posee acceso completo al sistema. Previsto para el sistema completo

9.2 Aplicación del Proceso Unificado de Desarrollo (PUD)

Para el desarrollo del sistema se adopta el **Proceso Unificado de Desarrollo (PUD)** debido a que permite organizar el proyecto mediante un enfoque iterativo e incremental, facilitando la incorporación progresiva de funcionalidades y reduciendo los riesgos durante el desarrollo.

Fase	Actividades realizadas
Inicio	Definición del problema, alcance, objetivos y requerimientos del sistema.
Elaboración	Diseño de la arquitectura, casos de uso, modelo de datos y diagramas UML.
Construcción	Desarrollo del prototipo utilizando Java y MySQL e implementación de los módulos principales.
Transición	Pruebas funcionales, corrección de errores y preparación del sistema para su utilización.

9.3 Resultado del análisis

Como resultado de esta etapa se definió la arquitectura general del sistema, las entidades principales de la base de datos, los casos de uso que deberá implementar la aplicación y las tecnologías que serán utilizadas durante el desarrollo. Esta información constituye la base para las etapas de diseño, implementación y pruebas que se presentan en los apartados siguientes.

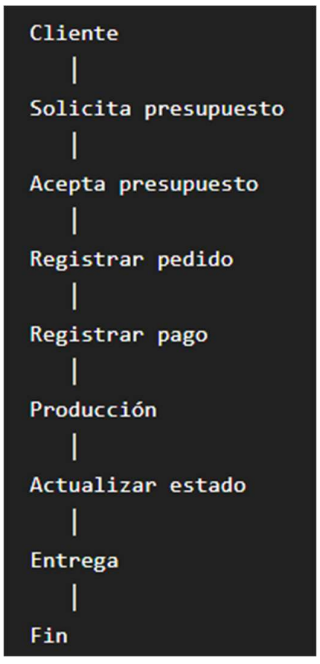


Figura 4. Flujo general del proceso de negocio.

10. Etapa de diseño

La etapa de diseño tuvo como finalidad definir la estructura técnica del sistema a partir de los requerimientos identificados durante el análisis. Para ello se estableció una arquitectura organizada en capas, se definieron las principales entidades del sistema y se diseñó la base de datos que permitirá almacenar la información de manera segura y consistente.

El sistema utilizará una arquitectura de tres capas, separando la interfaz de usuario, la lógica de negocio y el acceso a los datos. Esta organización facilita el mantenimiento del software, mejora su escalabilidad y permite incorporar nuevas funcionalidades sin modificar la estructura principal de la aplicación.

10.1 Arquitectura del sistema

La arquitectura propuesta está compuesta por los siguientes niveles:

Capa	Función
Presentación	Interfaz utilizada por los usuarios para gestionar clientes, pedidos, pagos y producción.
Lógica de negocio	Implementa las reglas del negocio, valida la información ingresada y coordina las operaciones del sistema.
Acceso a datos	Gestiona la comunicación con la base de datos MySQL mediante JDBC.
Base de datos	Almacena de forma permanente la información de clientes, pedidos y pagos mediante tablas relacionadas.

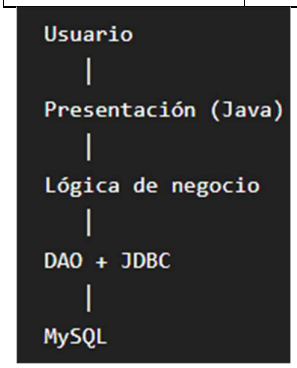


Figura 5. Arquitectura general del sistema. Fuente: elaboración propia.

10.2 Diseño de clases

Las principales clases identificadas durante el diseño son:

Clase	Responsabilidad
Cliente	Gestiona la información de los clientes.
Pedido	Administra los pedidos realizados por los clientes.
Pago	Registra los pagos asociados a cada pedido.
ClienteDAO	Realiza las operaciones sobre la tabla Cliente.
PedidoDAO	Gestiona el acceso a los datos de los pedidos.

PagoDAO	Administra los pagos almacenados en la base de datos.

Los diagramas de casos de uso, clases y entidad-relación presentados anteriormente constituyen la base del diseño detallado del sistema y serán utilizados durante la implementación del prototipo.

11. Etapa de implementación

La implementación del sistema se realizó utilizando Java como lenguaje de programación y MySQL como sistema gestor de base de datos, cumpliendo con los requisitos establecidos en la consigna. La comunicación entre ambos componentes se efectuó mediante JDBC, permitiendo ejecutar operaciones de alta, baja, modificación y consulta sobre la base de datos.

Con el objetivo de facilitar el mantenimiento del software, el proyecto se organizó siguiendo el patrón Modelo-Vista-Controlador (MVC), separando la interfaz de usuario, la lógica de negocio y el acceso a los datos.

11.1 Estructura del proyecto

El proyecto está organizado en los siguientes paquetes:

Paquete	Función
modelo	Contiene las clases que representan las entidades del sistema.
dao	Implementa las operaciones de acceso a la base de datos.
controlador	Coordina la interacción entre la interfaz y la lógica del negocio.
vista	Contiene las pantallas utilizadas por los usuarios.
util	Incluye clases auxiliares, como la conexión a MySQL.

11.2 Prototipo

Como prototipo operacional se implementaron las siguientes funcionalidades:

- Registro y consulta de clientes.
- Registro y consulta de pedidos asociados a clientes.
- Registro de pagos con validación del saldo disponible.
- Cálculo automático de saldos pendientes.
- Actualización de estados de producción.
- Búsqueda de pedidos mediante identificador.

El prototipo utiliza una interfaz de consola y mantiene la información en MySQL mediante JDBC. La autenticación de usuarios y la interfaz gráfica quedan fuera del alcance implementado.

El código fuente está administrado mediante Git y almacenado en un repositorio de GitHub, permitiendo controlar las versiones del proyecto y facilitar el trabajo colaborativo durante el desarrollo.

12. Etapa de pruebas

La etapa de pruebas tuvo como objetivo verificar el funcionamiento del prototipo y su integración con MySQL. Se realizaron pruebas funcionales sobre el registro de clientes, pedidos y pagos, la consulta de saldos, la actualización de estados y la búsqueda de pedidos. También se comprobaron las validaciones de importes y el comportamiento del programa ante operaciones inválidas. Los resultados y evidencias se detallan en las secciones 20 y 27.

12.1 Casos de prueba

Caso de prueba	Acción	Resultado esperado
Validación de pago	Registrar un importe superior al saldo pendiente	Operación rechazada y continuidad del menú
Registrar cliente	Ingresar datos obligatorios	Cliente registrado correctamente
Registrar pedido	Asociar un pedido a un cliente	Pedido almacenado en la base de datos
Registrar pago	Registrar un pago parcial	Saldo pendiente actualizado
Actualizar producción	Modificar el estado del pedido	Estado actualizado correctamente

Las pruebas se ejecutaron sobre el prototipo desarrollado en Java. La persistencia se verificó mediante consultas SQL directas sobre MySQL y la comparación de los resultados con los mostrados por la aplicación.

12.2 Resultado esperado

Las pruebas realizadas permitieron verificar las operaciones principales del prototipo y la consistencia entre los resultados mostrados en Java y los datos almacenados en MySQL. Se comprobó, además, el rechazo de pagos superiores al saldo pendiente sin finalizar inesperadamente la ejecución del programa.

13. Base de datos

La persistencia de la información se implementó mediante MySQL, utilizando tres tablas principales: cliente, pedido y pago. El modelo de datos fue diseñado siguiendo el modelo relacional, utilizando claves primarias y claves foráneas para garantizar la integridad referencial entre las distintas entidades.

La base de datos constituye uno de los componentes principales del sistema, ya que permite conservar la información de manera permanente y facilitar su consulta desde la aplicación desarrollada en Java.

13.1 Modelo relacional

Las entidades principales del sistema son las siguientes:

Entidad	Descripción
Cliente	Almacena los datos personales de cada cliente.
Pedido	Registra los pedidos realizados y su estado de fabricación.
Pago	Guarda los pagos efectuados por cada cliente para un pedido determinado.

Las relaciones entre las entidades son:

- Un Cliente puede tener muchos Pedidos.
- Un Pedido pertenece a un único Cliente.
- Un Pedido puede registrar muchos Pagos.

13.2 Normalización

El modelo de datos fue diseñado respetando los principios de normalización con el objetivo de reducir la redundancia y garantizar la consistencia de la información.

- Primera Forma Normal (1FN): todas las tablas poseen atributos atómicos y no existen grupos repetitivos.
- Segunda Forma Normal (2FN): todos los atributos dependen completamente de la clave primaria de su tabla.
- Tercera Forma Normal (3FN): se eliminaron dependencias transitivas, evitando la duplicación innecesaria de datos.

La aplicación de estas reglas permite mejorar el rendimiento de la base de datos y facilitar el mantenimiento del sistema.

13.3 Diagrama Entidad-Relación

El modelo entidad-relación presentado en la sección de diagramas representa las entidades principales del sistema y las relaciones existentes entre ellas.

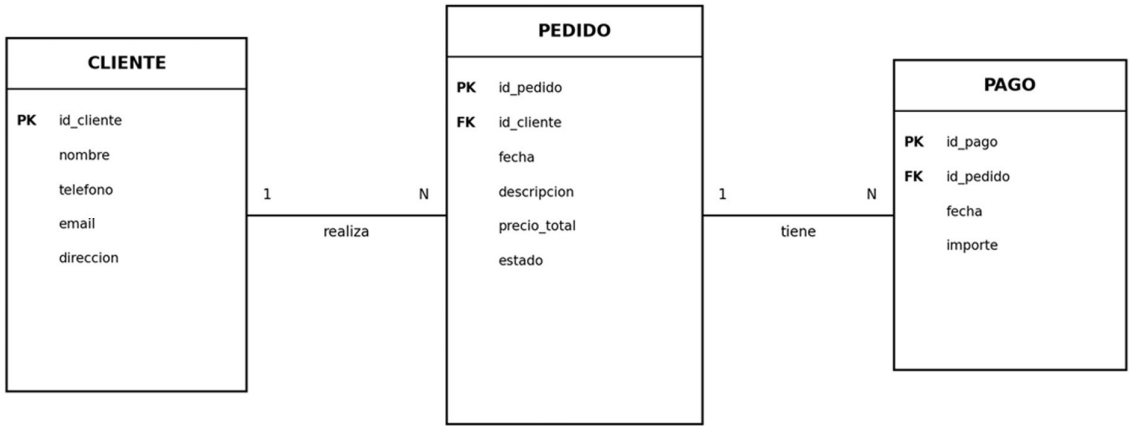


Figura 6. Modelo Entidad-Relación del sistema. Fuente: elaboración propia.

El diagrama muestra las relaciones Cliente (1:N) Pedido y Pedido (1:N) Pago, utilizando claves primarias y claves foráneas para garantizar la integridad referencial de la base de datos.

14. Implementación de la base de datos MySQL

La base de datos del sistema fue implementada utilizando MySQL, permitiendo almacenar de forma permanente la información correspondiente a clientes, pedidos y pagos. La estructura fue diseñada siguiendo el modelo entidad-relación presentado anteriormente, utilizando claves primarias y claves foráneas para garantizar la integridad referencial entre las tablas.

La implementación contempla la creación de la base de datos, la definición de las tablas principales y la ejecución de operaciones básicas de inserción, consulta y eliminación de registros.

14.1 Creación de la base de datos

```
CREATE DATABASE muebleria;

USE muebleria;
```

14.2 Creación de las tablas

Tabla Cliente

```
CREATE TABLE cliente (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    telefono VARCHAR(30),
    email VARCHAR(100),
    direccion VARCHAR(150)
);
```

Tabla Pedido

```sql
CREATE TABLE pedido (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT NOT NULL,
    fecha DATE NOT NULL,
    descripcion VARCHAR(255),
    precio_total DECIMAL(12,2),
    estado VARCHAR(30),
    fecha_entrega DATE,
    FOREIGN KEY (id_cliente)
        REFERENCES cliente(id_cliente)
);
```

Tabla Pago

```sql
CREATE TABLE pago (
    id_pago INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido INT NOT NULL,
    fecha DATE,
    importe DECIMAL(12,2),
    FOREIGN KEY (id_pedido)
        REFERENCES pedido(id_pedido)
);
```

14.3 Inserción de registros

Para verificar el funcionamiento de la base de datos se insertaron registros de prueba.

```sql
INSERT INTO cliente(nombre,telefono,email,direccion)
VALUES
('Juan Pérez','11-45678910','juan@gmail.com','Av. Rivadavia 1200'),
('María Gómez','11-43211234','maria@gmail.com','Mitre 350');
```

```sql
INSERT INTO pedido(id_cliente,fecha,descripcion,precio_total,estado,fecha_entrega)
VALUES
(1,'2026-09-10','Placard de melamina',950000,'En producción',NULL),
(2,'2026-09-15','Mueble de TV',1200000,'Confirmado',NULL);
```

```sql
INSERT INTO pago(id_pedido,fecha,importe)
VALUES
(1,'2026-09-10',300000),
(1,'2026-09-18',200000),
(2,'2026-09-15',500000);
```

14.4 Consultas SQL

Algunas consultas implementadas fueron las siguientes.

Listar clientes

```
SELECT * FROM cliente;
```

Mostrar todos los pedidos

```
SELECT * FROM pedido;
```

Pedidos de un cliente

```
SELECT c.nombre,
       p.descripcion,
       p.estado
FROM cliente c
INNER JOIN pedido p
ON c.id_cliente = p.id_cliente;
```

Pagos realizados

```
SELECT p.id_pedido,
       pa.fecha,
       pa.importe
FROM pedido p
INNER JOIN pago pa
ON p.id_pedido = pa.id_pedido;
```

Saldo pendiente

```
SELECT
    p.id_pedido,
    p.precio_total,
    COALESCE(SUM(pa.importe), 0) AS abonado,
    p.precio_total - COALESCE(SUM(pa.importe), 0) AS saldo
FROM pedido p
LEFT JOIN pago pa ON p.id_pedido = pa.id_pedido
GROUP BY p.id_pedido, p.precio_total;
```

14.5 Eliminación de registros

Para eliminar registros de la base de datos se utilizaron las siguientes instrucciones.

Eliminar un pago.

```
DELETE FROM pago
WHERE id_pago=3;
```

Eliminar un cliente.

```
DELETE FROM cliente
WHERE id_cliente = 2
AND NOT EXISTS (
    SELECT 1
    FROM pedido
    WHERE pedido.id_cliente = cliente.id_cliente
);
```

La eliminación de clientes se encuentra condicionada por la integridad referencial. Solo pueden eliminarse directamente aquellos clientes que no tengan pedidos asociados. Esta restricción evita la pérdida de consistencia de los datos.

15. Definiciones de comunicación

El sistema propuesto utilizará una arquitectura cliente-servidor, donde la aplicación desarrollada en Java actuará como cliente y el servidor MySQL será el encargado de almacenar y administrar la información de manera permanente.

La comunicación entre la aplicación y la base de datos se realizará mediante JDBC (Java Database Connectivity), que permite ejecutar consultas SQL desde Java y recuperar los resultados de forma segura y eficiente.

En cuanto a la infraestructura de red, el sistema podrá funcionar tanto en una red local (LAN) como a través de Internet, dependiendo de las necesidades de la empresa. Para ello se empleará el protocolo TCP/IP, utilizado para el intercambio de información entre la aplicación y el servidor donde se encuentre alojada la base de datos.

15.1 Protocolos y tecnologías utilizadas

Componente	**Tecnología utilizada**
Lenguaje de programación	Java
Base de datos	MySQL
Conector de base de datos	JDBC
Arquitectura	Cliente - Servidor
Protocolo de comunicación	TCP/IP
Red	LAN / Internet
Control de versiones	Git y GitHub

15.2 Funcionamiento de la comunicación

El usuario interactúa con la aplicación Java mediante una interfaz de consola. Las operaciones se gestionan a través de las clases de acceso a datos (DAO), que ejecutan consultas y actualizaciones sobre MySQL mediante JDBC. Los resultados obtenidos se presentan nuevamente en la consola, permitiendo consultar clientes, pedidos, pagos y saldos.

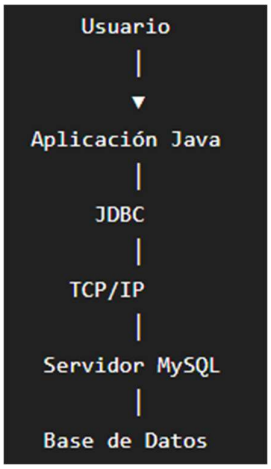


Figura 7. Esquema general de comunicación. Fuente: elaboración propia.

15.3 Seguridad

El prototipo utiliza credenciales específicas para acceder a MySQL. La contraseña se obtiene mediante la variable de entorno MUEBLERIA_DB_PASSWORD, evitando incorporarla directamente al código fuente. Las claves primarias, claves foráneas y validaciones contribuyen a mantener la integridad de los datos. La autenticación de usuarios de la aplicación queda prevista para una futura versión. Se recomienda implementar copias de seguridad periódicas.

16. Conclusiones de la AP2

El desarrollo del presente proyecto permitió aplicar las distintas etapas del Proceso Unificado de Desarrollo para el análisis, diseño e implementación de un sistema de gestión orientado a una mueblería dedicada a la fabricación de muebles a medida.

La solución propuesta responde a la problemática identificada durante el relevamiento inicial, permitiendo centralizar la información de clientes, pedidos, pagos y producción en una única base de datos. La utilización de Java y MySQL proporciona una plataforma sólida para el desarrollo del prototipo, mientras que la arquitectura propuesta facilita el mantenimiento y la incorporación de futuras funcionalidades.

Finalmente, la implementación de una base de datos relacional, junto con las consultas SQL y las pruebas realizadas, demuestra la viabilidad técnica del proyecto y sienta las bases para su evolución hacia un sistema completo de gestión empresarial.

17. Desarrollo del prototipo en Java

La tercera actividad profundiza la etapa de construcción del Proceso Unificado de Desarrollo (PUD). Se implementó un prototipo operacional de consola en Java, conectado a MySQL mediante JDBC, que permite registrar y consultar clientes y pedidos, registrar pagos, calcular saldos y actualizar estados de producción. Se conserva el alcance y el modelo de datos de las actividades anteriores.

Repositorio de código y archivos asociados: https://github.com/LautaroFR/seminario-ap1-muebleria

17.1 Organización técnica

Paquete	Responsabilidad

modelo	Clases Persona, Cliente, Empleado, Pedido y Pago; atributos y reglas del dominio.
dao	ClienteDAO, PedidoDAO y PagoDAO: consultas y actualizaciones SQL.
util	Conexion y PruebaConexion: conexión y verificación JDBC.
controlador	Main: punto de entrada de la aplicación. Gestion y Prueba: componentes de lógica y pruebas desarrollados en AP3.
vista	ConsolaVista: interfaz de consola que presenta el menú, recibe las entradas y muestra los resultados de las operaciones.

La aplicación utiliza MySQL 8.0, la base muebleria y las tablas cliente, pedido y pago, vinculadas por claves foráneas. Las operaciones JDBC emplean un usuario de aplicación con permisos limitados. La contraseña se obtiene de la variable de entorno MUEBLERIA_DB_PASSWORD y no se incorpora al repositorio.

18. Aplicación de programación orientada a objetos

Pilar	Implementación concreta
Encapsulamiento	Los atributos se mantienen privados y se accede a ellos mediante métodos de las clases de modelo; las validaciones protegen estados inválidos.
Herencia	Cliente y Empleado extienden la clase Persona.
Abstracción	Persona es abstracta y declara mostrarInformacion(), que debe implementar cada subtipo.
Polimorfismo	Cliente y Empleado implementan mostrarInformacion() con resultados propios; pueden tratarse como Persona.

Los constructores inicializan los objetos con datos válidos. Los tipos utilizados incluyen int para identificadores, String para textos, BigDecimal para importes monetarios, LocalDate para fechas y enum para los estados del pedido. BigDecimal evita los errores de redondeo habituales de los tipos de coma flotante en cálculos financieros.

18.1 Clases y relaciones

Persona (abstracta) → Cliente y Empleado (herencia). Cliente 1:N Pedido; Pedido 1:N Pago (asociaciones del negocio y relaciones persistidas mediante claves foráneas). Los DAO encapsulan las operaciones SQL. En la versión final, Main inicia la aplicación y delega la interacción con el usuario a ConsolaVista, que presenta el menú y coordina las operaciones del prototipo. El diseño amplía el diagrama de clases de la AP2 sin modificar el modelo relacional básico.

19. Menú, estructuras de control y excepciones

La clase ConsolaVista implementa el menú interactivo mediante estructuras repetitivas y condicionales. El menú permanece disponible hasta que el usuario selecciona la opción 0. Las estructuras de selección permiten ejecutar las operaciones correspondientes y las colecciones se recorren para mostrar clientes y pedidos. La búsqueda se realiza mediante el identificador del pedido.

Opción	Operación
1–2	Registrar y listar clientes.
3–4	Registrar y listar pedidos.
5–6	Registrar pago y consultar saldo.
7–8	Actualizar estado y buscar pedido.
0	Salir del programa.

El tratamiento de errores contempla SQLException para problemas de acceso a MySQL, IllegalArgumentException para entradas o estados inválidos y NoSuchElementException para entradas no disponibles. Se informa el error al usuario sin terminar el ciclo principal. Al abrir la aplicación se verifica primero la conexión a la base de datos.

20. Pruebas de funcionamiento y evidencia

Las pruebas automatizadas iniciales del prototipo en memoria informaron resultados correctos para cliente, pedido, pago, saldo, estado, validación y polimorfismo. Además, se verificó manualmente la conexión JDBC y se contrastaron las operaciones de la aplicación con consultas SQL directas sobre MySQL. Estas verificaciones no equivalen a una cobertura completa de pruebas.

Prueba	Evidencia observada	Resultado
Conexión JDBC	PruebaConexion mostró “Conexion exitosa con MySQL” y la base muebleria.	Correcto
Persistencia de cliente	El cliente Pepe (ID 1) figuró tanto en el menú como en SELECT * FROM cliente.	Correcto
Registro de pedido	Pedido ID 1, descripción Placard, importe $1.000.000, visible en Java y MySQL.	Correcto
Registro de pago	Pago ID 1 por $300.000 asociado al pedido 1, visible en MySQL.	Correcto
Cálculo de saldo	Importe $1.000.000 menos pago $300.000: saldo $700.000, coincidente en Java y SQL.	Correcto
Actualización de estado	El pedido pasó a En producción y el estado se consultó desde Java y MySQL.	Correcto

20.1 Capturas de ejecución

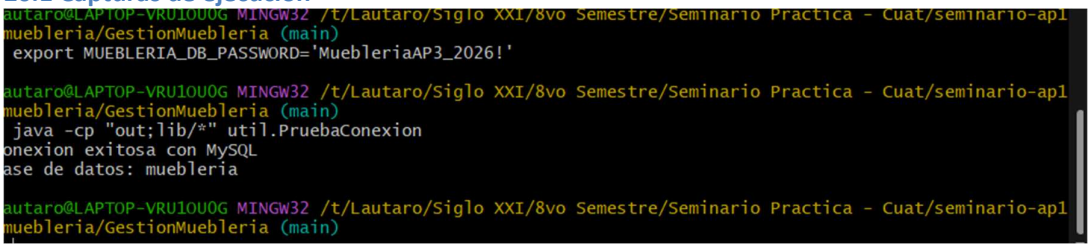


Figura 8. Verificación de conexión JDBC a la base muebleria. Fuente: elaboración propia (capturas de ejecución).

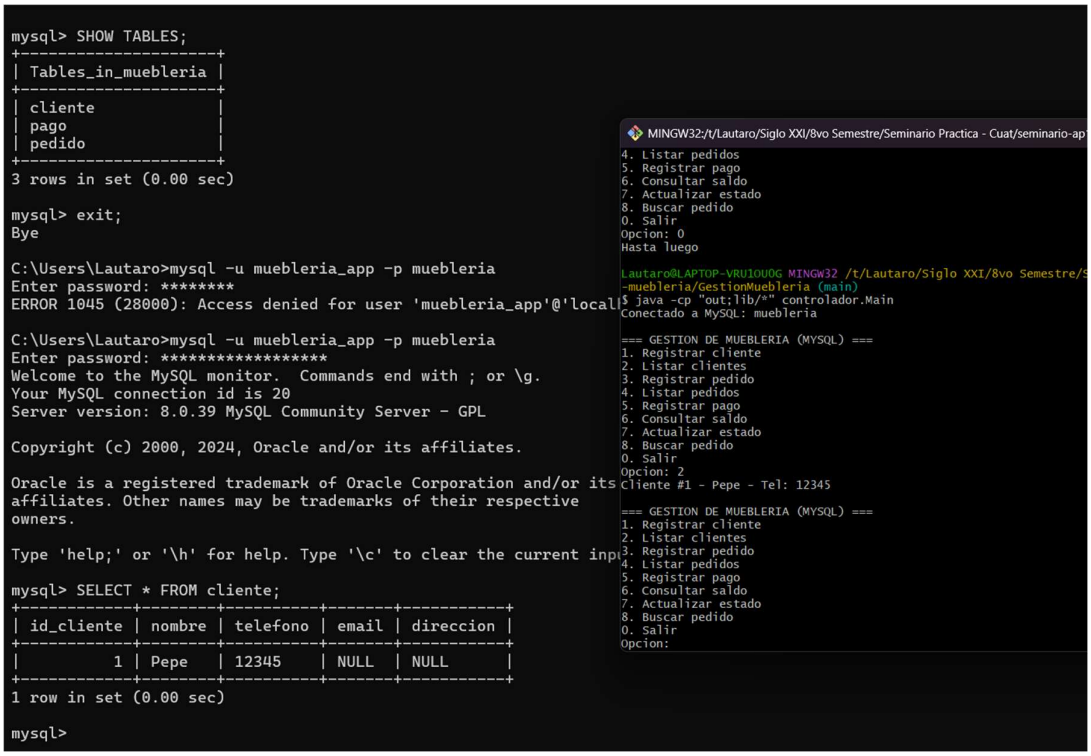


Figura 9. Persistencia de un cliente comprobada desde Java y MySQL. Fuente: elaboración propia (capturas de ejecución).

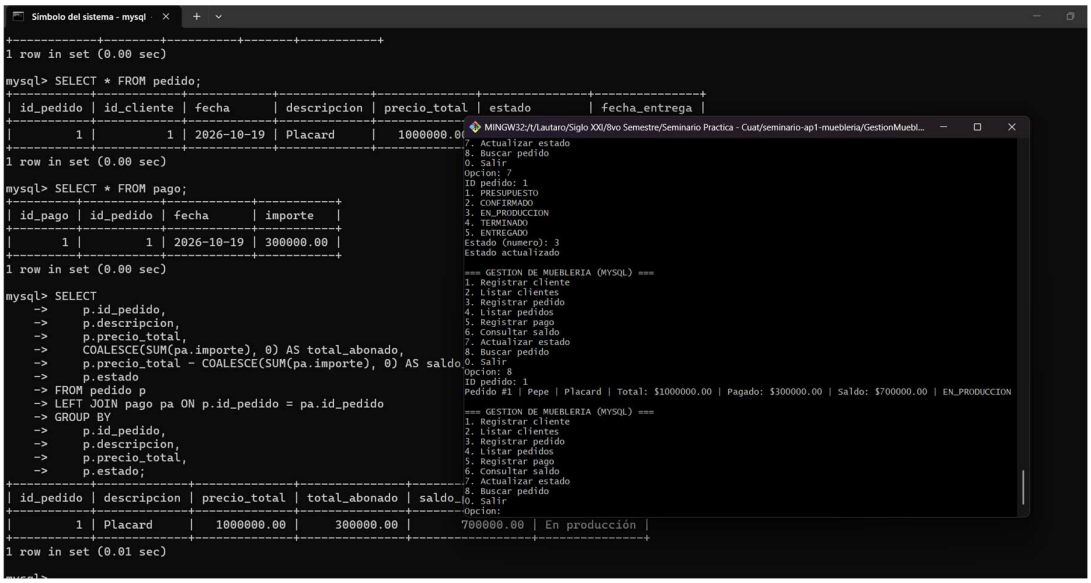


Figura 10. Consulta de pedido, pago, saldo y estado en Java y MySQL. Fuente: elaboración propia (capturas de ejecución).

20.2 Alcance y limitaciones

El prototipo operacional entregado implementa un menú por consola y operaciones persistentes para clientes, pedidos y pagos. No incluye todavía una interfaz gráfica funcional ni autenticación de usuarios, aunque ambas figuran en los requerimientos del sistema completo. En AP4 se incorporó ConsolaVista como interfaz de consola funcional, manteniendo la interfaz gráfica como posible ampliación futura. Las pruebas automatizadas en memoria y las verificaciones manuales con MySQL se documentan por separado.

21. Conclusiones de la AP3

La tercera actividad permitió evolucionar el diseño de las entregas anteriores hacia un prototipo Java operacional, aplicando encapsulamiento, herencia, polimorfismo y abstracción. El menú incorpora estructuras de control, constructores, validaciones y manejo de excepciones. La integración JDBC con MySQL permitió comprobar la persistencia de clientes, pedidos y pagos y el cálculo correcto del saldo pendiente. Como continuidad del PUD, quedan previstas la ampliación de pruebas, la autenticación y una interfaz gráfica.

22. Integración final del proyecto: AP4

La cuarta actividad consolida el prototipo operacional de gestión de pedidos y producción para una mueblería. Se mantiene el alcance definido en AP1–AP3 y se profundiza la arquitectura, el acceso a datos, la reutilización mediante interfaces y el empleo complementario de arreglos y ArrayList. El desarrollo corresponde a una nueva iteración de construcción y verificación del Proceso Unificado de Desarrollo (PUD).

23. Patrón de diseño seleccionado y justificación

Se adopta el patrón Modelo–Vista–Controlador (MVC), complementado por el patrón Data Access Object (DAO). MVC separa la representación de los datos, la interacción con el usuario y la coordinación de la ejecución. DAO encapsula las operaciones SQL y evita que la vista conozca los detalles de MySQL. Esta separación mejora la cohesión, facilita pruebas y permite sustituir la presentación por otra interfaz en futuras iteraciones.

Componente	Implementación
Modelo	modelo.Persona, Cliente, Empleado, Pedido y Pago; entidades y reglas del dominio.
Vista	vista.ConsolaVista; muestra el menú, lee entradas y presenta resultados.
Controlador	controlador.Main; punto de entrada que delega la ejecución a la vista. En este prototipo la vista aún coordina llamadas a DAO; es una aproximación ligera a MVC.
Persistencia (DAO)	dao.ClienteDAO, PedidoDAO y PagoDAO; operaciones JDBC con PreparedStatement.
Conexión	util.Conexion; DriverManager, credenciales desde variable de entorno.

24. Interfaces, abstracción y reutilización

La clase abstracta Persona conserva el contrato mostrarInformacion(), implementado por Cliente y Empleado. En la AP4 se agrega la interfaz genérica Repositorio<T>, que declara listar() y buscar(int). ClienteDAO y PedidoDAO implementan este contrato, permitiendo consultar diferentes entidades a través de una abstracción común. La clase PagoDAO mantiene una operación de registro especializada.

25. Arreglos y ArrayList complementarios

En la vista se utiliza Pedido.Estado[] estados = Pedido.Estado.values() para presentar las opciones de estado disponibles. El arreglo tiene una cantidad fija de elementos y se recorre por índice para asociar un número del menú con cada valor enumerado. Por otro lado, los métodos listar() de ClienteDAO y PedidoDAO devuelven ArrayList<Cliente> y ArrayList<Pedido>, respectivamente. Su tamaño es dinámico y depende de las filas recuperadas de MySQL. De esta forma se utilizan ambas estructuras en tareas diferentes y justificadas.

26. Persistencia MySQL y gestión de excepciones

La aplicación utiliza JDBC y el conector MySQL Connector/J para establecer conexiones a la base muebleria. Los DAO implementan inserciones, consultas y actualización de estados. Las sentencias preparadas parametrizadas disminuyen el riesgo de inyección SQL. El uso de try-with-resources asegura el cierre de Connection, PreparedStatement y ResultSet. Para registrar pagos se emplea una transacción con bloqueo del pedido, validación de saldo, commit y rollback ante errores.

Las excepciones SQLException se propagan hasta la interfaz para informar fallos de conexión o consulta; IllegalArgumentException permite señalar importes, estados o entradas inválidas; NoSuchElementException identifica registros inexistentes. No se incluyen contraseñas en el código: se obtiene MUEBLERIA_DB_PASSWORD del entorno.

27. Pruebas y resultados

Prueba	Criterio de verificación
Compilación	Todas las clases Java compilan sin errores con JDK 17+.
POO	controlador.Prueba imprime OK en pruebas de modelo y polimorfismo.
Persistencia	Registrar y listar clientes, pedidos y pagos; contrastar con SELECT en MySQL.
Actualización	Modificar estado y comprobarlo con SELECT.
ArrayList	Listar varios registros y comprobar la cantidad variable.
Arreglo	Mostrar opciones del enum y rechazar índices fuera de rango.
Errores	Probar ID inexistente, pago superior al saldo y conexión fallida sin cerrar abruptamente el menú.

La tabla presenta los criterios utilizados para evaluar las funcionalidades del prototipo. Las pruebas funcionales documentadas incluyen la persistencia de clientes, la consulta de pedidos, el cálculo de saldos, la actualización de estados y el rechazo de pagos superiores al saldo disponible. No se afirma una cobertura exhaustiva de todos los casos posibles.

Pruebas funcionales de AP4 realizadas sobre MySQL: se recuperó el cliente Pepe (ID 1); se consultó el pedido 2 «Mesa Ratona», de $500.000, con pago registrado de $200.000 y saldo pendiente de $300.000; se actualizó su estado a EN_PRODUCCION. El intento de registrar un pago de $600.000 fue rechazado con el mensaje «El pago excede el saldo pendiente» y el programa continuó en el menú. Las siguientes capturas documentan estos resultados.

28. Alcance y limitaciones del prototipo final

El prototipo incluye menú de consola, persistencia de clientes, pedidos y pagos, cálculo de saldos, búsqueda y actualización del estado. No implementa autenticación ni interfaz gráfica. La aplicación utiliza una separación MVC simplificada y puede evolucionar hacia un controlador con casos de uso independientes de la vista. La lectura y escritura de archivos no se incorpora por ser opcional en esta actividad.

29. Presentación del desarrollo y video

La presentación audiovisual del proyecto está disponible en https://youtu.be/oCMaTWO5Kzo. El video muestra la organización del código y una demostración de las operaciones del prototipo. La duración es de aproximadamente cuatro minutos.

30. Conclusiones de la AP4

La AP4 integra las decisiones de análisis y diseño de las entregas anteriores en un prototipo Java con persistencia MySQL. La organización mediante MVC simplificado y DAO mejora la separación de responsabilidades; las interfaces y la clase abstracta favorecen la reutilización; y el uso complementario de arreglos y ArrayList responde a diferentes necesidades de representación. Las pruebas funcionales y de integración permiten validar progresivamente la solución siguiendo el PUD.

31. Evidencias de ejecución de AP4

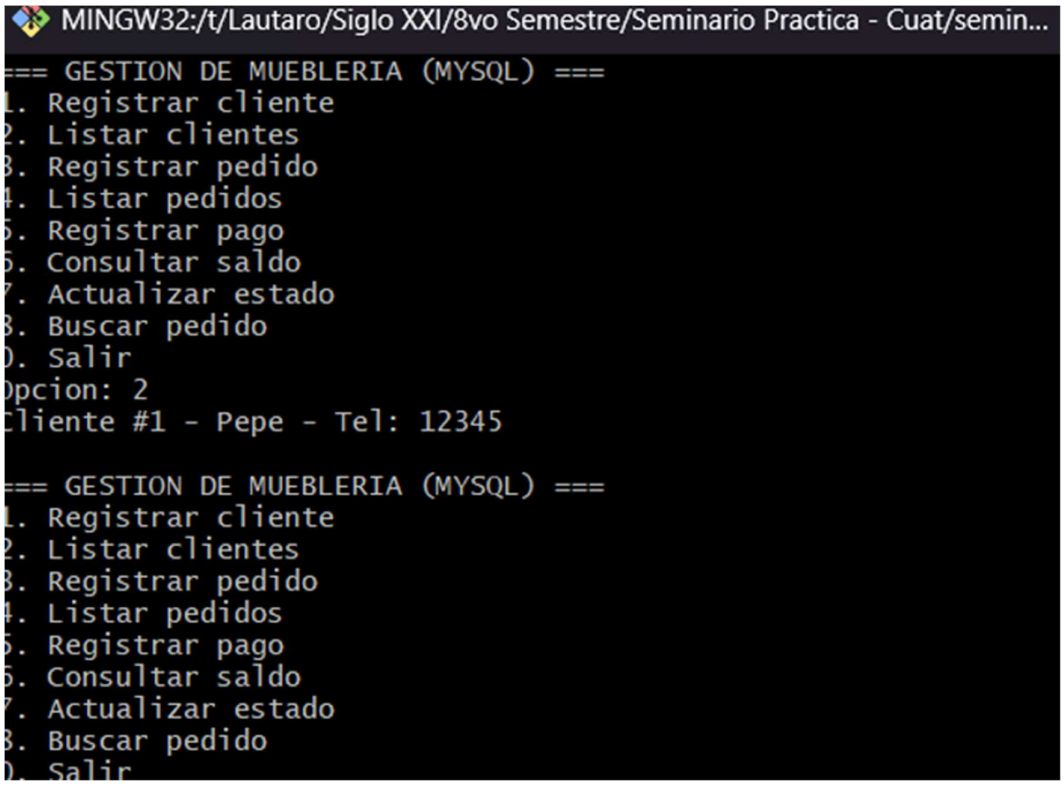


Figura 11. Consulta de clientes persistidos en MySQL

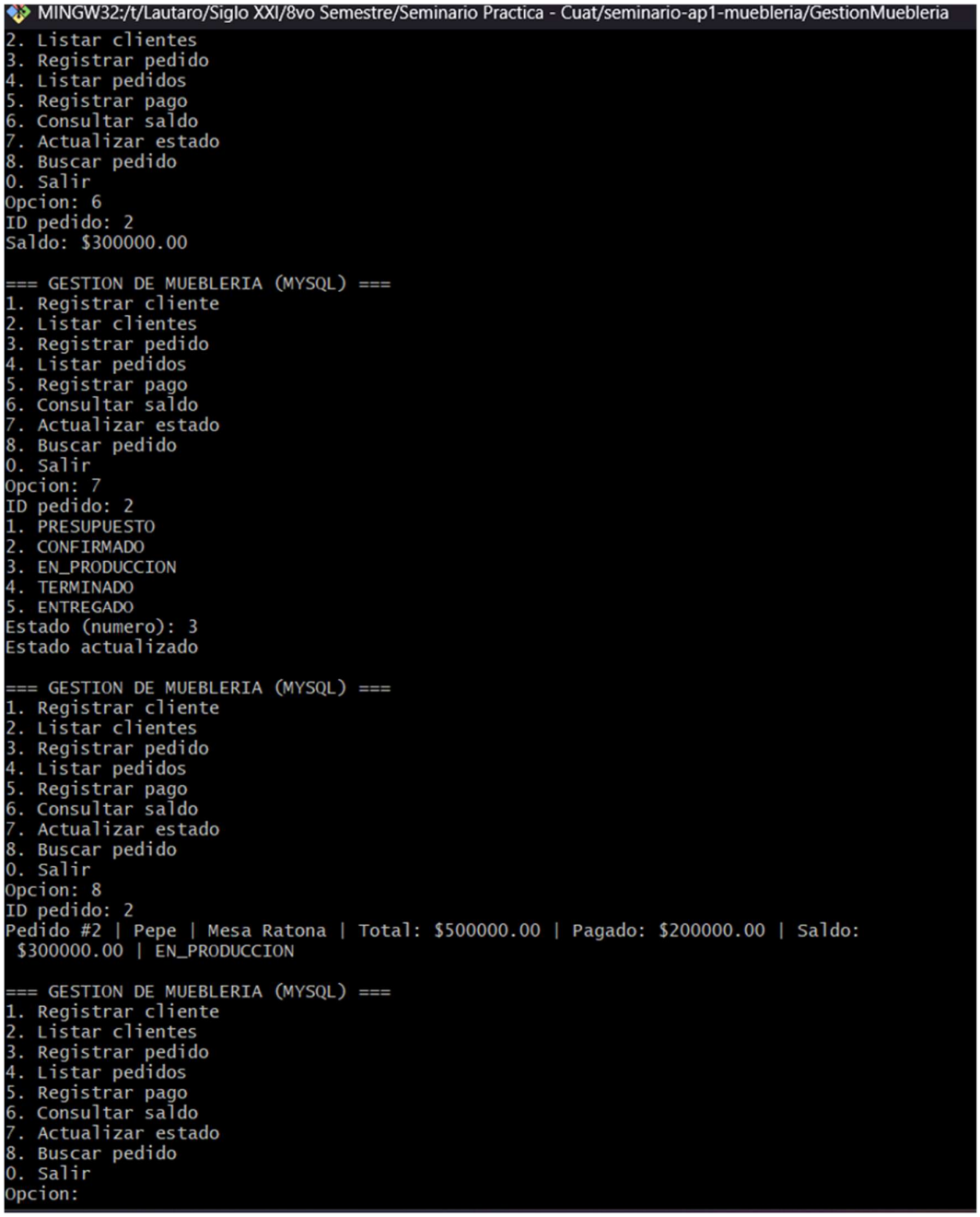


Figura 12. Saldo, actualización de estado y búsqueda de pedido

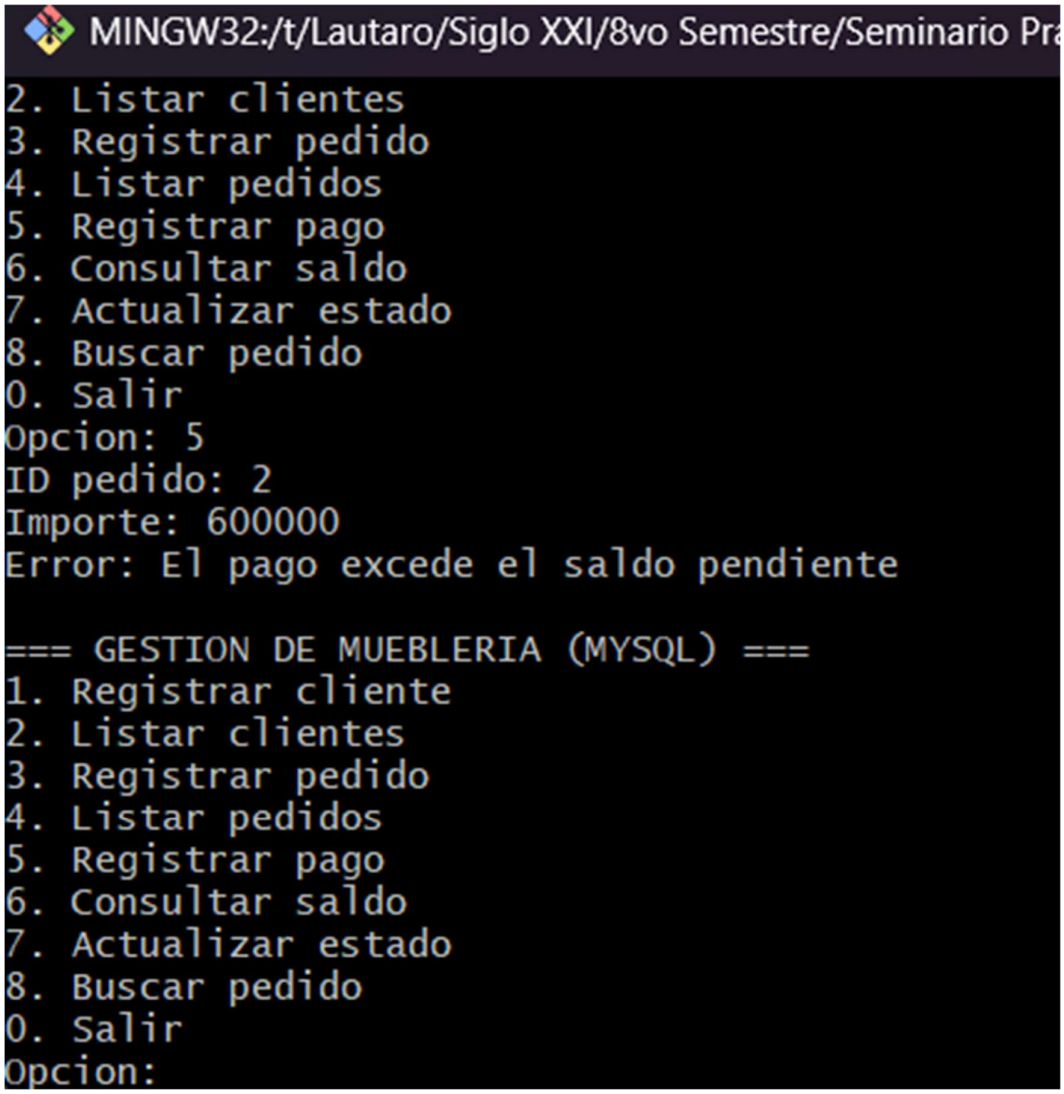


Figura 13. Manejo de excepción ante un pago superior al saldo

32. Diagrama de arquitectura del prototipo

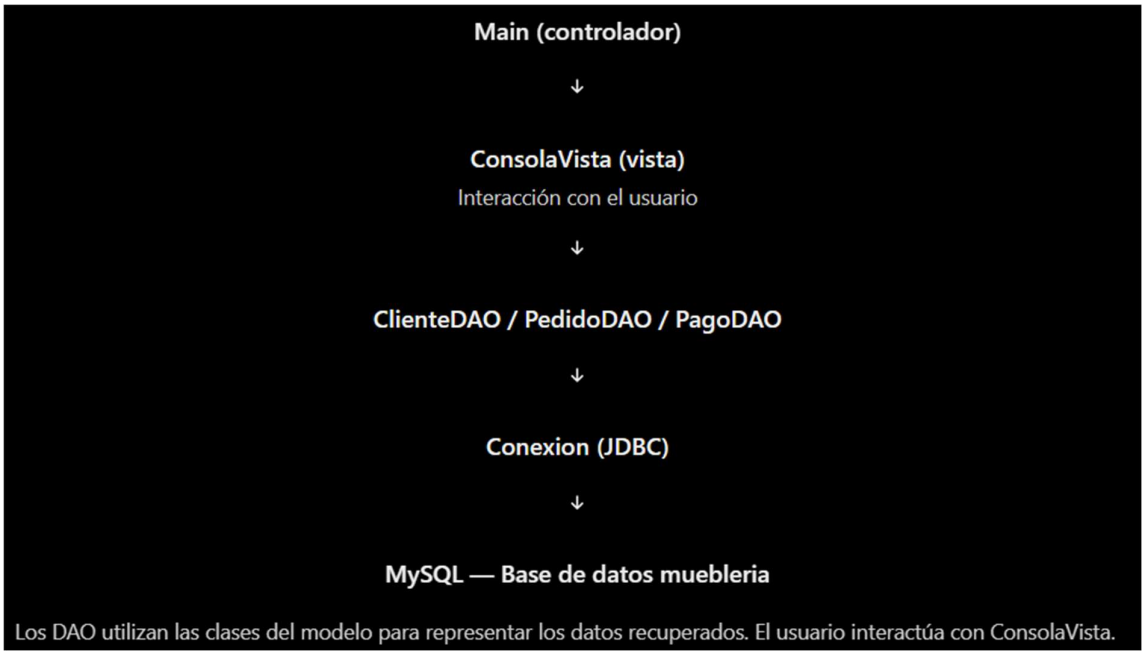


Figura 14. Arquitectura del prototipo Java con MVC simplificado y DAO. El punto de entrada Main inicia ConsolaVista; esta interactúa con los DAO, que utilizan JDBC para acceder a MySQL. Fuente: elaboración propia.

La interfaz Repositorio<T> define operaciones de consulta reutilizables, implementadas por los DAO correspondientes. La clase abstracta Persona permite especializar Cliente y Empleado. La enumeración de estados se recorre mediante un arreglo, mientras que las consultas devuelven listas dinámicas ArrayList.

33. Publicación y demostración final

El código fuente completo del proyecto, junto con los archivos asociados, se encuentra disponible en GitHub: https://github.com/LautaroFR/seminario-ap1-muebleria
La demostración del funcionamiento del prototipo, incluyendo la conexión a MySQL, la gestión de clientes y pedidos, el registro de pagos y las validaciones implementadas, se presenta en el siguiente video: https://youtu.be/oCMaTWO5Kzo
La presentación permite observar el funcionamiento del sistema desarrollado en Java y su integración con la base de datos MySQL.

34. Referencias

Elmasri, R., & Navathe, S. B. (2017). Fundamentos de sistemas de bases de datos (7.ª ed.). Pearson.

Kendall, K. E., & Kendall, J. E. (2011). Análisis y diseño de sistemas. Pearson Educación.

Oracle. (2024). Java Database Connectivity (JDBC) API. https://docs.oracle.com/

Oracle. (s. f.). Java Platform, Standard Edition Documentation. https://docs.oracle.com/en/java/javase/

Oracle. (s. f.). MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/en/

Saroka, R. (2002). Sistemas de información en la era digital. Fundación OSDE.

Sommerville, I. (2005). Ingeniería del software (7.ª ed.). Pearson-Addison Wesley.